

The Robot Adapter: Navigation Techniques in OVMM

Role of the Robot Adapter

`core/missions/search_and_fetch.py` defines a `RobotAdapter` protocol (`navigate_to`, `navigate_near`, `look_at`, `get_rgbd`, `grasp`) entirely in pure Python, with no ROS dependency. The mission logic – “go to a candidate location, look, detect, grasp, repeat” – is written once against that protocol. `ros2_ws/src/fetcher/fetcher/robot_adapter.py` (`HsrRobotAdapter`) is the concrete implementation for the HSR: it is the only place where Nav2, TF, camera topics, and joint trajectories actually appear. This seam keeps the mission runner testable with a dry-run mock and reusable if the robot platform ever changes.

Frame handling. The scene graph stores ground-truth poses in Gazebo’s `world` frame; Nav2 plans in the `map` frame, which is anchored at the robot’s spawn pose, not at `world (0,0)`. Every `navigate_to`/`navigate_near`/`look_at` call stamps its input as `world` and lets a `tf2` buffer convert it, rather than assuming the two frames coincide.

Navigation: Reachability-Aware Approach Poses

The first navigation approach picked a nav goal a fixed `standoff` (0.8 m) directly along $-X$ from the target’s centroid, regardless of what was actually there. This ignored both the environment (could land inside furniture, or facing a wall) and the manipulator (a deep shelf could put the object well beyond the arm’s working range, producing visibly overstretched grasps). The fix replaces the fixed offset with a *reachability-aware* search against Nav2’s live global costmap:

1. Fetch the global costmap (`BasicNavigator.getGlobalCostmap()`) and convert it to a plain `CostmapGrid` (a small dataclass with no ROS message dependency, so the search logic itself stays pure Python and unit -testable: `core/perception/active_perception.py`).
2. Around the target (c_x, c_y) , scan an annulus of candidate standoff points

$$(v_x, v_y) = (c_x + r \cos \theta_i, c_y + r \sin \theta_i), \quad r \in [r_{\min}, r_{\max}], \quad \theta_i = \frac{2\pi i}{n}$$

checking r closest-first, so the tightest reachable spot wins over a farther one.

3. Accept the first point whose costmap cost is below 253 (nav2.costmap_2d’s inscribed-obstacle threshold) – i.e. genuinely free, not merely “discouraged.” Its yaw faces back toward the target via `atan2`, so the nav goal itself doesn’t waste a step facing the wrong way.

This says nothing about how precisely Nav2 actually achieves that goal, though – real navigation has residual position/orientation error. That is handled separately, described next.

Listing 1: Core search, `active_perception.reachable_approach_pose`

```
1 for radius in np.linspace(reach_min, reach_max, n_radii):
```

```

2   for i in range(n_angles):
3       angle = 2 * math.pi * i / n_angles
4       vx, vy = cx + radius * math.cos(angle), cy + radius * math.sin(angle)
5       cost = costmap.cost_at(vx, vy)
6       if cost is not None and cost < COSTMAP_BLOCKED_THRESHOLD:
7           return _facing_pose(cx, cy, vx, vy)

```

$r_{\min}/r_{\max}$ are *not* a fixed pair of constants. An early version tied them to one global guess at the arm’s comfortable reach, which doesn’t generalize: a band measured from a target’s centroid is too close for furniture with real depth (an HSR ended up base-to-edge against a table in practice) and needlessly far for a small object. They are now computed per candidate in `core/missions/search_and_fetch.py`’s `_reach_band()` as a fixed clearance (`CLEARANCE.MIN/CLEARANCE.MAX`, roughly the robot’s own footprint radius) *plus* that candidate’s own `approach_radius` – half of the supporting furniture’s *shorter* horizontal dimension (`core/reasoning/scene_query.py`’s `_approach_radius()`), deliberately the shorter one since an approach is normally roughly perpendicular to a piece’s long axis. A small side table and a long counter each get a standoff scaled to their own size instead of one constant tuned for neither.

Graceful degradation. `navigate_near` falls back to the original fixed-standoff `approach_pose()` if the world→map TF lookup fails, the costmap service is unavailable, or every cell in the reach band is blocked – a mission step never hard-fails just because the costmap-aware path couldn’t run.

Active Perception, Re-centering, and the Grasp Handoff

Once parked, `look_at` points the head camera at the target using pure trigonometry in `head_geometry.py`: a pan delta from the bearing to the point in the current pan-link frame, and a tilt angle from its elevation in the base frame, both clamped to the joint limits read from the URDF. Both angles come from a live TF lookup of the robot’s actual current pose – not the nav goal it was aiming for – so `look_at` is already self-correcting for navigation error *within* a single call.

Re-centering on the actual detection. The first `look_at` still aims at the a-priori centroid (from the scene graph or the LLM), not at the object itself, so if navigation stopped a little off from the planned approach pose, that first SAM3 detection can land off-center or near the edge of the frame. `search_and_fetch._observe_and_detect` now treats that detection as provisional: it lifts the mask to a point cloud, takes its centroid, converts that camera-frame point back to world frame via a new adapter method (`localize_point`, a live `CAMERA_FRAME`→`WORLD_FRAME` TF lookup – the mirror image of `look_at`’s own conversion), and calls `look_at` a second time on *that* – the object’s real position, not the estimate. SAM3 runs once more on the re-centered view, and that second detection is what gets handed to `grasp()`. Either TF lookup or re-detection failing just falls back to the original detection rather than losing the object:

Listing 2: `search_and_fetch.py` – re-centering before grasping

```

1   points = np.asarray(mask_to_pointcloud(mask, depth, K).points)
2   center_world = robot.localize_point(points.mean(axis=0).tolist())
3   if center_world is None:
4       return None # caller falls back to the original detection
5   robot.look_at(center_world) # aim at the object itself, not the a-priori estimate
6   rgb2, depth2, K2 = robot.get_rgbd()
7   det2 = sam3.detect(rgb2, target)

```

After that, `grasp()` hands the (now centered) point cloud to `AnyGrasp`, transforms the chosen grasp pose from camera frame to `base_footprint` via a TF lookup taken at that exact moment, and converts

it to arm joints with the same kind of analytic formula (no IK solver) – AnyGrasp’s own candidate ranking already accounts for collisions, so the adapter only needs to reach the pose it is given.

Takeaway

The adapter’s real job is reconciling three coordinate conventions (world ground truth, Nav2 map, robot base/camera) and never fully trusting where it *meant* to end up: the costmap search turns an approximate target location into a reachable, collision-checked one instead of a hardcoded offset, and re-centering on the actual detection corrects for whatever residual error navigation still left, by looking again rather than assuming the first view was good enough. Both are the same principle – consult live data instead of trusting the plan – and it generalizes to any future grasp-region or approach-direction refinement.